UrbanPulse Project Specification

The goal is to build a system where citizens can report infrastructure issues like potholes or graffiti. To keep the system distributed, we're splitting the city into different zones, with each zone having its own regional server.

How it works

When a citizen opens the mobile app and submits a report, the app uses their GPS coordinates to figure out which regional server to contact. If a report is sent to the wrong server, that server will just forward the data to the correct one using a direct REST call. This keeps the data local to the area where the problem is while allowing the servers to stay in sync.

The AI Part

An AI agent will classify the incoming reports. When a photo and description are uploaded, the AI will automatically tag the category and check if the issue looks like a high-priority emergency. This helps the city admins see the most dangerous problems first without having to manually sort through every single ticket.

Technical Requirements

Backend & Data

- Servers: We'll use Node.js or Spring Boot for the regional servers. Each one will have its own database.
- Server-to-Server Sync: The servers will communicate directly via REST to hand off reports or sync status updates between zones.

Apps & UI

- Mobile: A Flutter or React Native app with access to the camera and high-accuracy GPS. It will include logic to route reports to the correct regional server IP based on location.
- Web: A React dashboard for admins, featuring a map interface (like Leaflet) to track and manage active reports.

AI & Environment

- AI Integration: We'll use the OpenAI or Google Vision API. The backend will send the report's photo and text to get an automated category and urgency flag.

- Deployment: The entire system will be containerized with Docker so we can run multiple regional nodes on one machine to show how the distributed setup works.